Another way we *could* get a container to run Rails is by taking that same set of commands needed to install Rails' requirements, and chain them together into a long, compound `docker run` command. However, not only would the command be huge, ugly, and hard to remember, but worst of all, it would be *slow*. The setup instructions would have to be run from scratch every time, including installing Node.js or our gem dependencies. We don't want to have to be waiting minutes just to start a Rails server.

Compound docker run Commands



You may be curious to know how you'd run multiple commands in a container with `docker run`. The problem is that `docker run` is designed to start a container and run a *single* command.

The trick to get around this limitation is to use the `bash` command's `-c` option, which starts a Bash shell and immediately executes whatever you pass in as a string. This lets you do the following:

```
$ docker run <options> [image:version] \
    bash -c "command1 && command2 && command3..."
```

Clever old `bash`.

So what's the *real* solution?

Well, we want to be able to run containers based on a *preconfigured* state that has everything needed in order to run Rails. What we really want is a factory—hint hint—for creating these perfect Rails server containers. Did you get the subtle hint? In the previous chapter, we said that an *image* is a "factory for creating specific containers." Sounds like just what we need.

Defining Our First Custom Image

In real life, a factory doesn't come out of nowhere. It has to be constructed from blueprints: detailed plans and instructions that describe exactly what it's supposed to look like. Docker's container factories—in other words, images—are no different. They require a special blueprint file aptly named a `Dockerfile`. A `Dockerfile` uses special syntax to describe exactly how the image should be constructed. If you've heard the expression *infrastructure as code*, this is an example of it: a `Dockerfile` describes how a machine image is configured as shown in the figure on page 19.

A `Dockerfile` is made up of various *instructions*—such as `FROM`, `RUN`, `COPY`, and `WORKDIR`—each capitalized by convention. Rather than talk about them in the abstract, though, let's look at a specific example.